libuv & the Thread Pool

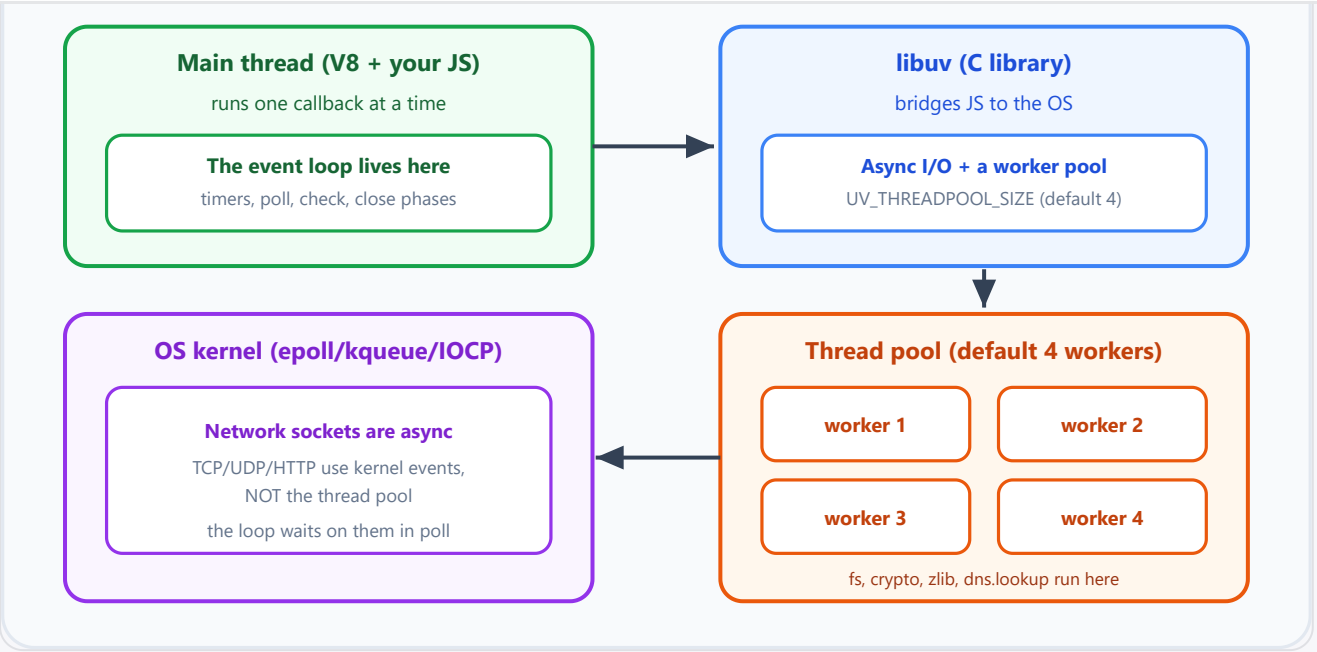
Why it matters

"Node.js is single-threaded" is the most repeated — and most misleading — sentence in backend interviews. Your *JavaScript* runs on a single thread. Node as a whole is emphatically multi-threaded: underneath your code sits **libuv**, a C library that owns the event loop *and* a pool of worker threads, plus it hands as much work as possible straight to the operating system's own asynchronous machinery. Understanding this layer is what lets you answer the real questions: why does reading 4 files concurrently sometimes serialise into two batches? Why does `crypto.pbkdf2` scale to exactly 4 parallel calls and then stall? Why is a DNS lookup mysteriously slow while an HTTP request to the same host is fast?

Every one of those has the same root: some operations are truly async at the OS level and cost no thread, while others are only *faked* as async by running them on libuv's small thread pool. Knowing which is which is the difference between an application that scales and one that intermittently freezes under load for reasons nobody can explain.

The concept, simply

Picture Node as three cooperating layers. At the top, **V8** runs your JavaScript on one thread. In the middle, **libuv** is the plumbing: it runs the event loop on that same main thread, and it also manages a separate **thread pool** (four background worker threads by default). At the bottom is the **operating system kernel**, which has its own efficient event notification systems (`epoll` on Linux, `kqueue` on macOS/BSD, IOCP on Windows).



One JS thread runs your code; libuv gives Node real concurrency via the OS kernel and a worker thread pool.

When your code does something asynchronous, libuv routes it down one of two paths:

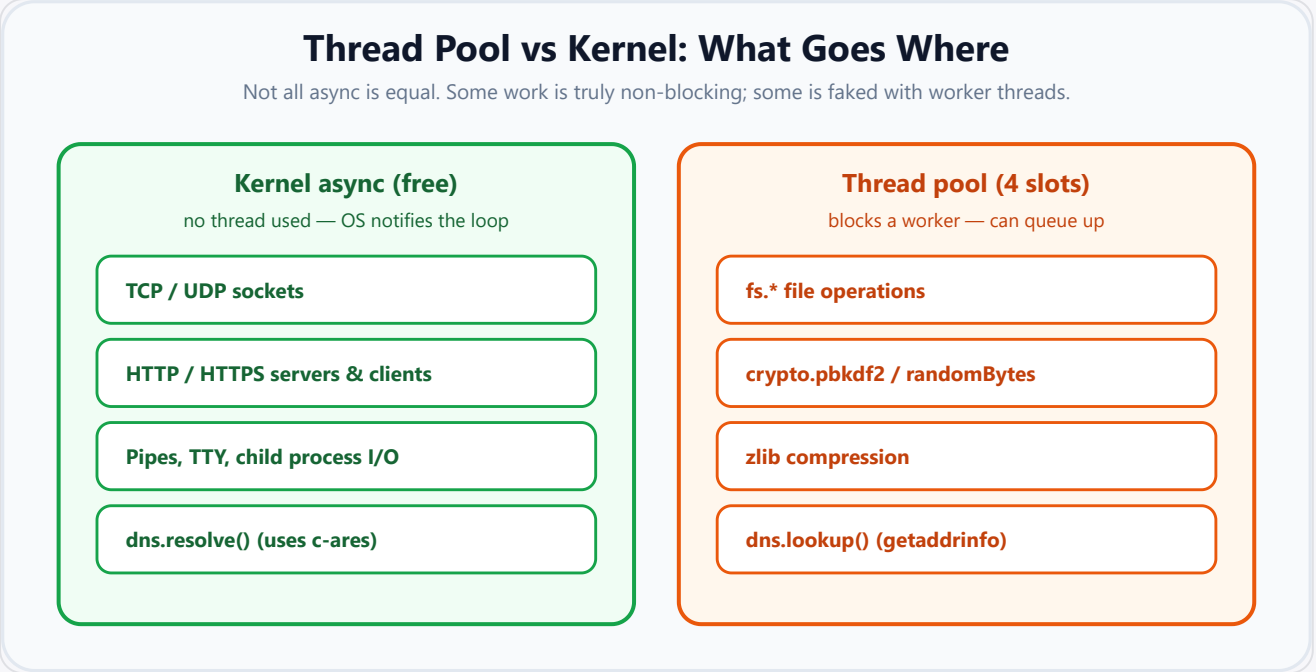
- **The kernel path (free concurrency):** network sockets — TCP, UDP, HTTP, HTTPS — are natively asynchronous in the OS. libuv just registers interest and the kernel notifies the poll phase when data arrives. *No thread is consumed.* You can have tens of thousands of idle connections for almost no cost.
- **The thread pool path (borrowed concurrency):** some operations have no async kernel API, or their API is blocking. The filesystem is the classic case. For these, libuv hands the work to one of its worker threads, which performs the blocking call and then posts the result back to the loop. Because there are only four workers by default, only four such operations run truly in parallel.

How it works

What actually uses the thread pool. The list is short and worth memorising because it is exactly what interviewers probe:

- **File system** — almost all of `fs.*` (`readFile`, `writeFile`, `stat`, etc.). The POSIX file API is synchronous, so libuv offloads it.

- **zlib** — asynchronous compression/decompression (gzip, deflate, brotli).
- `dns.lookup` — because it calls the system resolver `getaddrinfo`, which is blocking. This is the sneaky one: it is what `http.get('http://example.com')` uses under the hood to resolve the hostname.



Network I/O rides the kernel for free; file, crypto, zlib and dns.lookup borrow one of four worker threads.

What does NOT use the thread pool. All network I/O. A thousand concurrent HTTP requests do not consume a thousand threads — they are multiplexed through the kernel and serviced in the poll phase. Confusingly, `dns.resolve()` (which speaks DNS directly via the bundled c-ares library over a socket) is kernel-async and does *not* touch the pool, while `dns.lookup()` (the default that `getaddrinfo` backs) *does*. Same-sounding functions, opposite thread behaviour.

The pool has exactly four slots by default. The environment variable `UV_THREADPOOL_SIZE` controls it (max 1024, and it must be set before the pool is created — practically, before you do any pool work). If you fire five `crypto.pbkdf2` calls at once, four start immediately and the fifth waits in a queue until a worker frees up. This produces a very recognisable performance signature: latency that is flat up to four concurrent operations, then rises in steps.

Why the "single-thread myth" is half true and half dangerous. It is true that your JS logic never runs on two threads at once — you get memory safety for free, no locks, no data races on your objects. It is dangerous when people conclude "so Node can't do parallel work". It can: I/O is parallel via the kernel, and heavy filesystem/crypto/zlib work is parallel via the pool. What is *not* parallel is your own CPU-bound JavaScript — a `for` loop crunching numbers occupies the one JS thread and blocks the loop (see the blocking topic, and use `worker_threads` for genuine JS parallelism).

Code examples

Example 1 — the four-slot ceiling is observable.

JAVASCRIPT Copy

```
const crypto = require('crypto');
const start = Date.now();

// Fire six identical CPU-bound hashes concurrently.
for (let i = 1; i ≤ 6; i++) {
  crypto.pbkdf2('secret', 'salt', 100000, 64, 'sha512', () ⇒ {
    console.log(`hash ${i} done in ${Date.now() - start}ms`);
  });
}

// Typical output (4-core machine, default pool of 4):
//   hash 1 done in ~180ms
//   hash 2 done in ~180ms
//   hash 3 done in ~180ms
//   hash 4 done in ~180ms   ← first four finish together
//   hash 5 done in ~360ms
//   hash 6 done in ~360ms   ← next two wait for a free worker
```

The first four run in parallel on the four workers; calls five and six queue and only start once a worker is free — so they finish in roughly double the time. This is the thread pool made visible.

Example 2 — raise the ceiling with `UV_THREADPOOL_SIZE`.

JAVASCRIPT Copy

```
// Now all six hashes above start immediately and finish together,
// because there are enough workers. Set it BEFORE any pool work;
// setting process.env.UV_THREADPOOL_SIZE at runtime is too late
// once the pool has been created.
```

Example 3 — network I/O ignores the pool entirely.

JAVASCRIPT Copy

```
const https = require('https');
const start = Date.now();

// 50 concurrent requests — served via the kernel, not 50 threads.
for (let i = 0; i < 50; i++) {
  https.get('https://example.com', (res) => {
    res.on('data', () => {});
    res.on('end', () => console.log(`req ${i}: ${Date.now() - start}ms`));
  });
}
// All 50 progress concurrently regardless of UV_THREADPOOL_SIZE;
// the pool default of 4 does not cap network concurrency.
```

Example 4 — the hidden DNS cost.

JAVASCRIPT Copy

```
const dns = require('dns');

dns.lookup('example.com', () => {});   // uses a thread-pool worker
dns.resolve4('example.com', () => {}); // uses c-ares over a socket, no wo

// Under heavy fs load, dns.lookup can queue behind file operations
// on the pool. If DNS feels slow, dns.resolve() often sidesteps it.
```

Common mistakes & gotchas

- **Believing all async work is thread-pool work.** Network I/O is kernel-async and does not touch the pool; assuming otherwise leads people to raise `UV_THREADPOOL_SIZE` hoping to serve more HTTP connections — it does nothing for that.
- **Assuming the pool is unlimited.** Four workers by default. Heavy concurrent `fs` + `crypto` + `zlib` compete for the same four slots and can serialise.
- **Forgetting `dns.lookup` uses the pool.** A burst of outbound HTTP can flood the pool with `getaddrinfo` calls and slow down unrelated file reads.
- **Setting `UV_THREADPOOL_SIZE` too late.** It must be set in the environment before the first pool operation; mutating `process.env` mid-run after the pool exists has no effect.
- **Thinking `worker_threads` and the libuv thread pool are the same thing.** They are not. The pool is internal to libuv for I/O-ish native work; `worker_threads` are separate V8 isolates you use to run *your own* CPU-bound JavaScript in parallel.
- **Expecting `fs` to be "free" like sockets.** Filesystem calls are blocking under the hood; concurrency is bounded by the pool, not the disk.

Best practices

- **Match `UV_THREADPOOL_SIZE` to your workload,** not blindly to CPU count. If you do heavy concurrent `fs` / `crypto` / `zlib`, a larger pool (e.g. 8–16) can help; measure before and after.
- **Use `dns.resolve()` instead of `dns.lookup()`** when you need many name resolutions and want to keep them off the pool.
- **Push genuine CPU-bound JavaScript to `worker_threads`** (or a separate process), never onto the libuv pool — the pool is for native I/O-style work.
- **Watch for the "flat then stepped" latency signature** under load; it usually means you have saturated the pool.
- **Do not conflate connection scaling with pool size.** To serve more sockets, tune the app and OS limits — the pool is irrelevant to network concurrency.

⭐ Interview lens

Your JavaScript executes on a single main thread, so there are no data races on your objects. But Node itself is multi-threaded: libuv runs a thread pool (4 by default) for blocking native work like `fs` and `crypto`, and network I/O is handled asynchronously by the OS kernel. So "single-threaded" describes JS execution, not the whole runtime.

Q: Which operations use the libuv thread pool?

Filesystem (`fs.*`), CPU-heavy crypto (`pbkdf2`, `scrypt`, `randomBytes`, key generation), `zlib`, and `dns.lookup` (because it wraps blocking `getaddrinfo`). Network sockets do *not* — they are kernel-async.

Q: You run 6 `pbkdf2` calls concurrently and the last two are twice as slow. Why?

The default pool has 4 workers. The first four run in parallel; the fifth and sixth queue until a worker frees up, so they complete in roughly double the time. Raising `UV_THREADPOOL_SIZE` to ≥6 fixes it (given enough CPU cores).

Q: Does increasing `UV_THREADPOOL_SIZE` let my server handle more concurrent HTTP connections?

No. HTTP/TCP is serviced via the kernel in the poll phase and does not consume pool threads. The pool size only affects `fs` / `crypto` / `zlib` / `dns.lookup` throughput.

Q: What is the difference between `dns.lookup` and `dns.resolve`?

`dns.lookup` calls the OS resolver `getaddrinfo`, which is blocking, so it runs on the thread pool. `dns.resolve` (and its variants) queries DNS servers directly via c-ares over a socket — kernel-async, no pool thread. Under heavy pool load, `lookup` can queue while `resolve` stays fast.

No. The libuv pool is internal, runs native I/O-style work, and you cannot put arbitrary JS on it. `worker_threads` are separate V8 isolates you create explicitly to run your own CPU-bound JavaScript in parallel, communicating via message passing or `SharedArrayBuffer`.

Cheat-card

- **1 JS thread** (V8) + **libuv** (event loop + thread pool) + **OS kernel**.
- **Kernel-async, no thread:** TCP/UDP/HTTP sockets, pipes, `dns.resolve`.
- **Thread pool (4 default):** `fs.*`, `crypto.pbkdf2/scrypt/randomBytes`, `zlib`, `dns.lookup`.
- Tune with `UV_THREADPOOL_SIZE` (max 1024); set it before any pool work.
- Pool ≠ network concurrency. Raising it won't serve more sockets.
- Real JS parallelism → `worker_threads`, not the pool.
- "Single-threaded" = JS execution only; the runtime is multi-threaded.

QUICK CHECK — no marks, just for you.

1. What is the default size of libuv's thread pool?

- ◯ 1
- ◯ 4
- ◯ 8
- ◯ Equal to the number of CPU cores

Check answer

2. [illegible] `crypto.pbkdf2` calls with the default pool, why do the last two finish roughly twice as slowly?

- pbkdf2 gets slower each time it runs
- The sixth call depends on the fifth completing
- Only four workers exist, so calls five and six queue until a worker frees up
- Crypto operations are always serialized to one thread

Check answer

3. Does raising `UV_THREADPOOL_SIZE` let a server handle more concurrent HTTP connections?

- Yes, HTTP requests each consume a pool thread
- Yes, but only for HTTPS
- Only if you also raise the timers limit
- No — network I/O is kernel-async and does not use the pool

Check answer

Previous
The Event Loop Phases

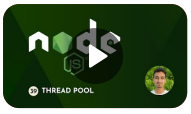
Next
process.nextTick vs setImmediate

LEARN MORE

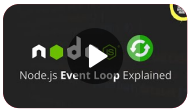
ARTICLES & DOCS

Don't Block the Event Loop (or the Worker Pool)
nodejs.org

VIDEOS



Node.js Tutorial - 39 - Thread Pool
Codevolution



This is how the Node.js Event Loop really works
Software Developer Diaries

Hand-picked references for this lesson.